

LoRA Meets the Global Workspace: Why Low-Rank Adapters Work Out of the Box

A personal writeup connecting Anthropic's "Verbalizable Representations Form a Global Workspace in Language Models" (Transformer Circuits, 2026) to the empirics of LoRA and other PEFT methods.

Part I: What the Paper Shows

"Verbalizable Representations Form a Global Workspace in Language Models" (Transformer Circuits, 2026) asks whether language models have something like the global workspace of cognitive science: a small, shared, broadcast-everywhere blackboard where the currently "in mind" contents live, distinct from the vast automatic machinery that does most of the processing.

The paper's central instrument is the **J-lens**. For each layer ℓ , define the averaged Jacobian from that layer's residual stream to the final-layer stream:

$$J_\ell = \mathbb{E} \left[\frac{\partial h_{\text{final}}}{\partial h_\ell} \right]$$

with the expectation taken over token positions and a large prompt corpus. Composing with the unembedding gives the J-lens:

$$\text{lens}(h) = \text{softmax}(W_U \cdot \text{norm}(J_\ell h)).$$

The rows of $W_U J_\ell$ form a dictionary of **J-lens vectors**, one direction per vocabulary token, per layer. The **J-space** at a layer is the set of activations expressible as sparse, non-negative combinations of a small number (~ 25) of these vectors.

The headline findings:

It's tiny. The J-space carries under 10% of the residual stream's activation variance, and at any moment only on the order of ~ 25 concept directions are meaningfully active. The other $\sim 90\%$ of the variance is automatic processing — syntax, bookkeeping, low-level features — that the model handles without "thinking about it."

It's mechanically privileged. The model's own MLPs amplify J-space-aligned directions roughly $10\times$ relative to random directions, and a small set ($\sim 1\%$ of attention heads) acts as dedicated broadcast machinery, copying J-space content across token positions. The network is pre-wired to detect, amplify, and distribute whatever lands in this subspace.

It's causally decisive. Swapping only the J-space component of an intermediate concept — a component holding just 10–15% of a probe's variance — flips the model's downstream answers, plans, and multi-hop conclusions on the majority of trials, while swapping the entire $\sim 90\%$ -variance remainder flips far fewer, and even that residual effect routes through the J-space. Behavioral control per unit of activation norm is concentrated in this tiny frame.

It's selectively necessary. Ablating the entire J-space leaves automatic capabilities — parsing, classification, span extraction — largely intact, while crippling flexible, deliberate behavior. The workspace occupies a middle band of the network, roughly layers 38–92 on a 0–100 scale, flanked by "sensory" early layers and "motor" late layers.

It's trainable. In a counterfactual reflection-training experiment, behavioral improvements from training were carried by concepts implanted into the workspace — and ablating those implanted representations reverted the behavioral gains. Small numbers of added, removable directions carried the learned behavior.

In short: a language model's flexible behavior is steered from a sparse, low-dimensional, amplified, broadcast subspace, while the bulk of its computation is automatic and lives elsewhere.

Part II: The LoRA Connection

Rewriting LoRA in the workspace's language

A LoRA adapter on any weight matrix computes

$$h_{\text{out}} = W_0 x + \frac{\alpha}{r} B A x.$$

Read it mechanically rather than as "a low-rank matrix": the rows of A are r **read vectors** detecting directions in the incoming residual stream, and the columns of B are r **write vectors** injecting directions back into it. Every LoRA adapter, wherever attached, is a small bank of learned read→write rules operating on the shared residual stream — the same stream the workspace lives in.

The deep link: LoRA's gradient *is* the J-lens Jacobian

This is the load-bearing step, and it falls straight out of the definitions. The gradient that trains a LoRA adapter at layer ℓ under next-token cross-entropy is, by the chain rule:

$$\nabla_{\Delta W} L \propto \left(\frac{\partial h_{\text{final}}}{\partial h_{\ell}} \right)^{\top} W_U^{\top} \frac{\partial L}{\partial \text{logits}} x^{\top}.$$

The term

$$\left(\frac{\partial h_{\text{final}}}{\partial h_{\ell}} \right)^{\top} W_U^{\top}$$

is the transpose of exactly the object the J-lens is built from: J_{ℓ} is the *expectation of this same Jacobian over a corpus*. This can be made rigorous (full derivation in the Appendix), and the structure of the result matters:

1. **Exact, assumption-free part.** Because standard LoRA initializes $B = 0$, everything B ever becomes is built from accumulated gradients. Pure chain rule then shows that under (S)GD, the column space of the trained ΔW is confined to the span of **pullback error vectors** — per-example Jacobians applied to $W_U^{\top}$ times the logit errors. Since cross-entropy errors are near-sparse, with mass on the target token and a few competitors, each write direction is approximately a **sparse combination of a few per-example J-lens-type vectors**, which is precisely the J-space's defining sparse-cone structure, not merely a linear span.

2. **Conditional part.** Replacing per-example Jacobians with their corpus mean J_ℓ requires a Jacobian-concentration assumption: fluctuations bounded by some δ . Under it, $\text{col}(\Delta W)$ lies within $O(\delta)$ of the span of J-lens vectors. Crucially, δ is not a rug to sweep things under — it is the theory's **testable dial**: small δ is the regime where LoRA matches full fine-tuning (steering tasks), large δ is where it fails (skill acquisition). Measuring δ on instruction data vs. code data is the single most decisive number for this whole story.
3. **A 's rows (read directions) are shaped by the input statistics $x^\top$ of the fine-tuning domain** — they become domain/context detectors. At initialization, with $B = 0$, A receives zero gradient: early training is purely B growing inside the pullback span through the fixed random read map.

So a trained LoRA adapter, to first order, learns rules of the form: *when the domain context is detected (A), write domain-appropriate, J-lens-aligned content into the stream (B)*. That is, almost literally, "repopulate the workspace with this domain's concepts." The model then does the heavy lifting for free: MLPs amplify J-aligned writes $\sim 10\times$ and broadcast heads relay them across positions, so a rank-8 nudge fans out into a behavioral change of much larger effective rank downstream.

This also dissolves the expressivity puzzle. Theory (Zeng & Lee, 2023) shows the worst-case rank needed to approximate an arbitrary target model is large — so LoRA's success was never about expressivity. LoRA works because it writes into a subspace the network is pre-wired to amplify and distribute. It doesn't need to be expressive; it needs to hit the bus.

Reinterpreting the follow-up literature

Intruder dimensions (Shuttleworth et al., "LoRA vs. Full Fine-Tuning: An Illusion of Equivalence," 2024/2025). LoRA-trained weight matrices acquire new, high-ranking singular vectors nearly orthogonal to the pretrained spectrum — full fine-tuning does not produce them — and forgetting is causally localized to these dimensions: scale down their singular values and pretraining behavior largely returns, with minimal target-task loss. Under the workspace reading, an intruder dimension is exactly what a "bolt a new read→write rule onto the stream" solution should look like spectrally: a new entry added to the broadcast dictionary rather than a reorganization of existing circuits. The parallel with the workspace paper's own reflection-training result is striking — there, ablating implanted workspace concepts reverted the behavioral improvement; here, downscaling intruder dimensions reverts LoRA's shift. Same signature, one in weight space, one in activation space: behavior change carried by a small number of added, removable directions.

LoRA learns less and forgets less (Biderman et al., 2024). LoRA matches full fine-tuning on instruction tuning but substantially underperforms on continued pretraining for code and math, where full fine-tuning's weight perturbations are $10-100\times$ higher rank than typical LoRA configs; MLP modules need higher rank than attention modules; middle layers need more rank than first and last. Map onto the workspace: instruction tuning is *steering* — changing which concepts load into the ~ 25 -slot, sub-10%-variance workspace and how they're used — a genuinely low-dimensional operation. Learning code semantics is modifying *automatic processing* in the other $\sim 90\%$, which no rank-16 hook can re-plumb; hence high-rank MLP updates. Even the layer profile matches: the rank demand peaks in the middle of the network, which is where the workspace band ($\approx L38-92$) sits, flanked by low-rank "sensory" and "motor" ends.

Why Q/K/V placement barely matters

The empirical fungibility of adapter placement — my old JPMC observation — has a clean explanation here. W_Q , W_K , W_V , W_O , MLP-in, and MLP-out all read from and write to the **same shared residual stream**, and the workspace lives in that stream. If the operation fine-tuning needs is "read broadcast context, write a steering direction," any hook inside the workspace layer band can implement it. The modules are different nonlinear wrappers around the same bus; LoRA placement is choosing which port to solder onto a shared backplane.

One real asymmetry survives, and it is provable, not just intuitive: Q and K only affect the bilinear attention scores, and because attention weights sum to one, a Q/K adapter's reachable perturbations of the stream lie in the span of **differences of OV vectors already present in the context**. Q/K adapters can re-route existing content but cannot write a single new direction into the stream. V , O , and the MLPs, by contrast, inject new directions freely. This is the rigorous version of "routing vs. content," and it explains the full empirical pattern: placement is roughly fungible when the content a task needs already exists in the context's OV image (typical steering tasks — hence my JPMC observation), while Q-only and K-only lag in the original paper's own ablation, and attention-only lags badly on hard domains. Targeting MLP modules turns out to be crucial for LoRA to approach full fine-tuning accuracy — satisfying, because the MLPs are precisely where the paper locates the $10\times$ workspace amplification.

The CNN analogy, upgraded

My old intuition was that LoRA placement resembles the CNN design question: abstract feature extraction first, meaning later, with the interesting choice being how far into abstract space you go before reading meaning out. The workspace paper gives that intuition a literal address inside a transformer. Early layers are "sensory" feature extraction; the middle workspace band is the abstract space where a sparse set of concepts is composed, amplified, and broadcast; late layers are the "motor" readout that converts workspace contents into tokens. LoRA on a steering task is, in effect, a learned re-labeling of what enters and exits that abstract space — which is why it behaves like swapping the FC head of a CNN rather than retraining the conv stack, and why it fails when the task actually requires new conv filters (new automatic skills like code and math).

Part III: The Direct Answer

Is LoRA's out-of-the-box success explained by the workspace being reconfigured to the fine-tuning domain?

As a hypothesis: **yes, for the class of tasks where LoRA matches full fine-tuning** — instruction following, style, persona, format, domain steering. The story: A learns domain detectors; B learns writes that are gradient-biased toward the J-frame, because the gradient flows through the same Jacobian that defines the lens; the model's own amplification and broadcast machinery multiplies the rank- r nudge into a global behavioral shift; the workspace is repopulated with the domain's concepts.

And **no, for skill acquisition** — new code/math ability requires high-rank rewiring of automatic processing outside the workspace, which is exactly where LoRA measurably fails.

Part IV: Caveats and the Bridging Experiment

Epistemic honesty: this synthesis is an interpretation, not a claim made by either paper. The workspace results are on Claude-family models and concern activations; the LoRA results are mostly on Llama-class models and concern weights. Nobody has run the bridging experiment. But it is concrete and cheap:

1. Take trained LoRA adapters from two task families: steering-type (instruction/style/persona) and skill-type (code/math continued pretraining).
2. Decompose the columns of B against the J-lens frame at the adapter's layer. The paper's own sparse gradient-pursuit machinery does this decomposition. On open models, a poor-man's J-lens can be approximated with the logit lens, which the paper notes captures much of the same structure in later layers.
3. Measure J-alignment of the write directions, per layer.
4. Measure the Jacobian-concentration constant δ , the variance of per-example backward propagators around J_ℓ , separately on instruction-tuning data and on code/math data. The theory stands or falls on δ being small in the first case and large in the second — this is the single most decisive number in the whole story.

Predictions: steering adapters show high J-alignment concentrated in the middle workspace band; code-CPT adapters show low J-alignment spread through MLPs; and the intruder dimensions of steering adapters should **be** the J-aligned components — so ablating their J-space projection should revert the fine-tuned behavior the same way downscaling intruder singular values does.

If that experiment comes out as predicted, then the folk claim "LoRA finds the weights that matter" gets replaced by something sharper and more beautiful: **LoRA doesn't find special weights at all — it finds the workspace, because gradient descent through J_ℓ can't help but find it.**

Support vectors. Attention. And now this. Three times the math turned out to be simpler than the machinery around it.

Appendix: A Rigorous Derivation of the LoRA–Workspace Link

Setup. Let $W_U \in \mathbb{R}^{|V| \times d}$ be the unembedding, with rows $w_v^\top$. For example n , let $h_{\ell,t}^{(n)} \in \mathbb{R}^d$ be the residual stream at layer ℓ , position t . Define the per-example backward propagator

$$\Phi_{t \rightarrow t'}^{(n)} = \frac{\partial h_{L,t'}^{(n)}}{\partial h_{\ell,t}^{(n)}} \in \mathbb{R}^{d \times d},$$

so the J-lens matrix is its corpus/position average,

$$J_\ell = \mathbb{E}[\Phi],$$

and the J-lens vectors are

$$j_{\ell,v} := J_\ell^\top w_v.$$

Equivalently, the $j_{\ell,v}$ are the rows of $W_U J_\ell$, viewed as directions in the layer- ℓ stream. The loss is next-token cross-entropy, whose logit gradient is

$$\varepsilon_{t'} := p_{t'} - e_{y_{t'}},$$

a near-sparse prediction-error vector. LayerNorm Jacobians are absorbed into Φ and W_U .

Attach LoRA to a direct-write matrix, such as the MLP down-projection or W_O . The W_V case adds a fixed linear post-map and goes through identically. The stream receives

$$\delta h_{\ell,t} = s B A x_t, \quad s = \frac{\alpha}{r}, \quad B_0 = 0,$$

with A_0 random.

Lemma 1 (gradient structure). The loss sensitivity to the write at (ℓ, t) is the pullback error vector

$$\left| \quad u_t^{(n)} = \sum_{t'} \left(\Phi_{t \rightarrow t'}^{(n)} \right)^\top W_U^\top \varepsilon_{t'}^{(n)} \succcurlyeq \sum_{t'} \sum_v \varepsilon_{t',v}^{(n)} \left(\Phi_{t \rightarrow t'}^{(n)} \right)^\top w_v. \right.$$

This gives

$$\left| \quad \nabla_B L = s \sum_{n,t} u_t^{(n)} \left(A x_t^{(n)} \right)^\top, \right.$$

and

$$\left| \quad \nabla_A L = s B^\top \sum_{n,t} u_t^{(n)} \left(x_t^{(n)} \right)^\top. \right.$$

Hence

$$\left| \quad \text{col}(\nabla_B L) \subseteq \text{span}\{u_t^{(n)}\} : \right.$$

every update to the write matrix is a combination of per-example pullbacks of unembedding rows, weighted by logit errors. At initialization,

$$\left| \quad \nabla_A L = 0, \right.$$

since $B_0 = 0$, so early training is purely B growing inside the pullback span through the fixed random read map A .

Lemma 2 (span preservation). Under plain (S)GD from $B_0 = 0$, induction gives

$$\left| \quad \text{col}(B_K) \subseteq U := \text{span}\{u_t^{(n)} \text{ over training}\}. \right.$$

Since

$$\left| \quad \Delta W = s B A, \right.$$

we have

$$\left| \quad \text{col}(\Delta W) \subseteq \text{col}(B) \subseteq U. \right.$$

That is, the trained adapter can only write directions from U into the stream.

Assumption A (Jacobian concentration). On the fine-tuning distribution,

$$\Phi_{t \rightarrow t'}^{(n)} = c_{t,t'}^{(n)} J_\ell + E_{t,t'}^{(n)}, \quad \|E_{t,t'}^{(n)}\|_{\text{op}} \leq \delta.$$

This is the one substantive assumption — an empirical property, not a theorem.

Theorem (mean-field alignment). Under Lemma 2 and Assumption A,

$$u_t^{(n)} = J_\ell^\top W_U^\top \tilde{\varepsilon}_t^{(n)} + r_t^{(n)},$$

with

$$\|r_t^{(n)}\| \leq \delta \sum_{t'} \|W_U^\top \varepsilon_{t'}^{(n)}\|,$$

where

$$\tilde{\varepsilon}_t^{(n)} = \sum_{t'} c_{t,t'}^{(n)} \varepsilon_{t'}^{(n)}.$$

Every write direction of the trained adapter therefore lies within $O(\delta)$ of

$$\text{span}\{j_{\ell,v}\}_v.$$

As $\delta \rightarrow 0$,

$$\text{col}(\Delta W) \subseteq \text{span}\{j_{\ell,v}\}_v,$$

the J-frame.

Remark (non-vacuousness). Since $|V| \gg d$, the span of all J-lens vectors is generically $\mathbb{R}^d$, so two sharpening facts carry the content.

1. **Sparsity.** $\varepsilon_{t'}$ concentrates on the target token and a few competitors, so each u_t is approximately a sparse combination of a few J-lens vectors — precisely the J-space's defining sparse-cone structure, not merely its linear span.
2. **Spectral compression.** J_ℓ 's fast-decaying spectrum (the workspace carries $< 10\%$ of variance) compresses any pullback toward its top singular subspace.

Corollary 1 (why GD selects these directions). To first order, a write δ changes logits by

$$\Delta z \approx W_U J_\ell \delta.$$

Loss decrease per unit $\|\delta\|$ is maximized on the top right-singular subspace of $W_U J_\ell$. With the measured $\sim 10\times$ MLP gain on J-aligned directions entering Φ multiplicatively, J-aligned writes dominate per-norm leverage. Steepest descent prefers the amplified subspace before rank constraints even bind — rank- r LoRA truncates to the top r directions of an already J-concentrated set.

Corollary 2 (failure regime). LoRA underperforms exactly when the hypotheses break:

1. Heterogeneous per-example propagators (large δ) give $\{u_t\}$ high effective dimension, of which rank- r captures only the top r principal directions.
2. Tasks requiring changes to Φ itself (new automatic skills) are second-order in the weights along the whole path — outside the first-order write picture entirely.

Both predict high-rank, MLP-concentrated updates, matching the measured $10-100\times$ rank gap for code/math continued pretraining.

Proposition (routing vs. content). For an adapter on W_Q or W_K , the perturbation reaches the stream only through attention weights $\alpha_{t,t'}$. With the value path frozen,

$$\delta(\text{head out}_t) = \sum_{t'} \delta\alpha_{t,t'} W_O W_V x_{t'}, \quad \sum_{t'} \delta\alpha_{t,t'} = 0.$$

Thus reachable perturbations lie in the span of differences of OV vectors already present in the context. Q/K adapters re-route existing content; $V/O/\text{MLP}$ adapters write new content, because $\text{col}(B)$ is free within U . Placement fungibility holds precisely when the task's needed content already exists in the context's OV image — typical of steering tasks — and fails (Q-only, K-only, attention-only on hard domains) when it doesn't.

Caveats.

1. Lemma 2 holds for SGD/gradient flow; Adam's per-coordinate preconditioning breaks exact span preservation.
2. Assumption A is where rigor meets empirics; δ is measurable, not proven small, and the paper's J_ℓ is estimated on a general corpus rather than the fine-tuning distribution.
3. The argument is first-order (NTK-style): Φ drifts during training.

4. J_ℓ averages over positions; per-position propagators are retained here and averaged only inside Assumption A.

One-sentence summary. Because B starts at zero, everything it ever becomes is built from gradients; the gradient at layer ℓ is the per-example J-lens matrix applied to sparse logit errors; so the adapter's write directions are sparse combinations of J-lens vectors up to a Jacobian fluctuation term — and that fluctuation term is exactly the quantity that blows up on the tasks where LoRA is known to fail.

References

- Anthropic (2026). *Verbalizable Representations Form a Global Workspace in Language Models*. Transformer Circuits. <https://transformer-circuits.pub/2026/workspace/index.html>
- Hu et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685